

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 4

Duration: 1 Hour
Total: Marks:40

Annexure A

5

Code of Conduct:

I SR Kaarthika (192412294)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SR Kaarthika

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

- (a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.
- (b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).
- (c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

- (a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.
- (b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.
- (c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

- (a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.
- (b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

- (a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.
- (b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.
- (c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

INDUSTRY SCENARIO: DIABETES PREDICTION AND PERSONALIZED TREATMENT

Task 1: Data Preparation & Inductive Learning

(a) Inductive Learning Approach

Inductive learning learns a general pattern from previously observed patient records and uses it to predict the outcome for new patients.

For this problem, a **supervised classification approach** is suitable because historical patient records contain input features and a known diabetes outcome.

Target variable:

- Diabetes – 0 = Non-diabetic, 1 = Diabetic.

Important features:

1. **Glucose Level** – High glucose is one of the strongest indicators of diabetes.
2. **BMI** – Higher BMI is associated with increased risk of developing type-2 diabetes.
3. **Blood Pressure** – High blood pressure is commonly associated with metabolic disorders and diabetes risk.
4. **Age** – Diabetes risk generally increases with age.
5. **Family History** – A family history of diabetes can indicate increased genetic risk.

Thus, the model learns a relationship:

Patient Features → **Diabetes Risk/Diagnosis**

(b) Handling Class Imbalance

The dataset contains more non-diabetic patients than diabetic patients. Therefore, accuracy alone may be misleading.

A suitable approach is **SMOTE (Synthetic Minority Over-sampling Technique)**.

SMOTE creates synthetic examples of the minority diabetic class instead of simply duplicating existing records.

Advantages:

- Improves representation of diabetic cases.
- Helps the model learn the minority class.
- Reduces the risk of ignoring diabetic patients.

SMOTE should be applied **only to the training data**, not before the train-test split, to avoid data leakage.

For a medical application, **class weights** can also be considered because false negatives are clinically important.

(c) 70:30 Train-Test Split and Preprocessing

The dataset is divided as:

- **70% → Training data**
- **30% → Testing data**

A 70:30 split provides enough data for model training while retaining a sufficiently large independent test set for reliable evaluation.

For a medical application, the split should be **stratified** so that the diabetic/non-diabetic proportion is approximately maintained in both sets.

Preprocessing:

1. Missing numerical values → Median imputation.
2. Missing categorical values → Mode imputation.
3. Categorical features such as family history → Encoding.

4. Numerical features → Standardisation/normalisation.
5. Class imbalance → SMOTE on training data only.

Normalisation is particularly important for models such as Logistic Regression because features such as age, glucose and BMI have different numerical ranges. Decision Trees generally do not require normalisation.

Task 2: Decision Tree for Diagnosis

(a) Decision Tree Construction

A Decision Tree can be constructed using the **Gini Index** or **Information Gain**.

Example Python implementation:

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
```

```
dt = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    random_state=42
)
```

```
dt.fit(X_train, y_train)
```

```
plt.figure(figsize=(15, 8))
plot_tree(
    dt,
    feature_names=X_train.columns,
    class_names=["Non-Diabetic", "Diabetic"],
    filled=True
)
plt.show()
```

The first three levels can be visualised using `max_depth=3`.

Root Node Selection

The root node is selected as the feature that provides the greatest reduction in impurity.

For Gini Index:

$$\text{Gini} = 1 - \sum(p_i^2)$$

where p_i is the proportion of samples belonging to class i . For

Information Gain:

$$\text{Information Gain} = \text{Entropy}(\text{parent}) - \text{Weighted Entropy}(\text{children})$$

A feature such as **Glucose Level** is expected to be a strong root candidate because it can effectively separate diabetic and non-diabetic patients.

The exact root node and Gini/Information Gain values must be obtained from the actual dataset.

(b) Model Evaluation

```
from sklearn.metrics import accuracy_score, precision_score
from sklearn.metrics import recall_score, f1_score, confusion_matrix
```

```
y_pred = dt.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

The important measures are:

- **Accuracy** – Overall percentage of correct predictions.
- **Precision** – Percentage of predicted diabetic patients who are actually diabetic.
- **Recall** – Percentage of actual diabetic patients correctly identified.
- **F1-score** – Balance between precision and recall.

False Negatives

A **False Negative (FN)** occurs when:

Actual = Diabetic, Predicted = Non-Diabetic

False negatives are particularly serious in this application because a diabetic patient may remain undiagnosed and untreated.

To reduce false negatives:

- Increase the importance of the diabetic class using `class_weight`.
- Apply SMOTE to training data.
- Adjust the classification threshold where appropriate.
- Optimise specifically for recall/sensitivity.
- Use additional clinical testing for high-risk patients.

In a clinical screening system, higher **recall/sensitivity** is generally preferred because missing a diabetic patient can have serious consequences.

(c) Post-Pruning Using Cost Complexity

A fully grown Decision Tree can overfit the training data. Cost-complexity pruning reduces unnecessary branches.

```
path = dt.cost_complexity_pruning_path(X_train, y_train)
```

```
ccp_alphas = path.ccp_alphas
```

```
models = []
```

```
for alpha in ccp_alphas:
```

```
    model = DecisionTreeClassifier(
        criterion="gini",
        ccp_alpha=alpha,
        random_state=42
    )
```

```
    model.fit(X_train, y_train)
```

```
    models.append(model)
```

```
scores = [model.score(X_test, y_test) for model in models]
```

```
best_model = models[scores.index(max(scores))]
```

```
print("Best post-pruned accuracy:", max(scores))
```

Comparison

Model	Main Advantage	Main Problem
Pre-pruned Tree	Controls complexity during training	May underfit
Post-pruned Tree	Removes unnecessary branches after training	Requires selecting pruning parameter

For clinical deployment, the **pruned tree is generally preferable** if it maintains good recall and performance because it is simpler, less prone to overfitting, and easier to interpret.

However, accuracy alone should not determine deployment. Recall, calibration, clinical validation and safety should also be considered.

Task 3: Statistical Learning for Risk Stratification

(a) Logistic Regression

Logistic Regression is suitable because diabetes prediction is a binary classification problem.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, accuracy_score
```

```
lr = LogisticRegression(max_iter=1000)
```

```
lr.fit(X_train, y_train)
```

```
lr_pred = lr.predict(X_test)
```

```
lr_prob = lr.predict_proba(X_test)[:, 1]
```

```
print("Accuracy:", accuracy_score(y_test, lr_pred))
```

```
print("AUC-ROC:", roc_auc_score(y_test, lr_prob))
```

Comparison

The actual values should be obtained from the dataset:

Model	Accuracy	AUC-ROC
Decision Tree	Run code	Run code
Logistic Regression	Run code	Run code

Interpretation:

- Decision Trees can model nonlinear relationships and interactions.
- Logistic Regression provides a simple statistical relationship between predictors and diabetes probability.
- Logistic Regression is often preferable when **interpretability, probability estimates and stable relationships** are important.
- Decision Trees are useful when relationships are nonlinear and decision rules are required.

For a medical risk-stratification system, Logistic Regression can be attractive because its coefficients can help explain how individual risk factors influence predicted risk.

(b) Feature Importance Analysis

Decision Tree

```
importance = dt.feature_importances_
```

```
for feature, value in sorted(
    zip(X_train.columns, importance),
    key=lambda x: x[1],
    reverse=True
):
    print(feature, value)
```

The top three predictors can then be identified from the largest importance values.

A typical result may contain features such as:

1. Glucose Level
2. BMI
3. Age/Blood Pressure

The exact ranking depends on the dataset.

Logistic Regression

```
import pandas as pd
```

```
import numpy as np
```

```
coef = pd.Series(
    np.abs(lr.coef_[0]),
    index=X_train.columns
)
```

```
print(coef.sort_values(ascending=False))
```

The features with the largest absolute coefficients have the greatest influence on the model.

Do They Agree?

The two models may agree on major predictors such as **glucose, BMI and age**, but their rankings may differ.

This happens because:

- Decision Trees measure impurity reduction.
- Logistic Regression measures the strength of a linear relationship.
- Decision Trees can capture nonlinear relationships.
- Correlated features can affect Logistic Regression coefficients.

Therefore, feature importance should ideally be interpreted together with clinical knowledge rather than relying on one model alone.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) MDP Formulation

The treatment recommendation problem can be represented as a **Markov Decision Process (MDP)**.

States

Patient health can be represented using simplified states:

- S0 – Low Risk
- S1 – Moderate Risk
- S2 – High Risk
- S3 – Diabetic/Uncontrolled
- S4 – Improved/Controlled

The state can be determined from variables such as glucose, BMI and blood pressure.

Actions

The agent can select:

- A0 – Diet
- A1 – Exercise
- A2 – Medication
- A3 – Monitor

Reward Function

Example:

- Health improves → **+10**
- Health remains stable → **+2**
- Health worsens → **-5**
- Unsafe/undesirable transition → **-10**

The reward encourages the agent to select actions that improve patient health.

Transition Dynamics

After an action, the patient's health state changes.

For example:

High Risk + Exercise → Moderate Risk

or

Moderate Risk + Diet → Low Risk

The transition depends on the patient's current condition and the selected treatment action.

In a real medical system, these transitions must be learned from validated clinical data rather than assumed.

(b) Q-Learning

The Q-learning update equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Example hyperparameters:

- Learning rate, $\alpha = 0.1$
- Discount factor, $\gamma = 0.9$
- Exploration rate, $\epsilon = 0.1$

Initial Q-table:

State	Diet	Exercise	Medication	Monitor
Low Risk	0	0	0	0
Moderate Risk	0	0	0	0
High Risk	0	0	0	0
Uncontrolled	0	0	0	0

Example update:

Suppose:

$Q(\text{High Risk, Exercise}) = 0$

Reward = +10

Maximum future Q-value = 5

Then:

$$Q_{\text{new}} = 0 + 0.1[10 + (0.9 \times 5) - 0]$$

$$Q_{\text{new}} = 1.45$$

Another example:

If:

$Q(\text{Moderate Risk, Diet}) = 1$

Reward = +2

Maximum future Q-value = 4

Then:

$$Q_{\text{new}} = 1 + 0.1[2 + (0.9 \times 4) - 1]$$

$$Q_{\text{new}} = 1.46$$

These updates gradually increase the Q-values of actions that lead to better health states.

Training

The agent should be trained for at least **5 patient episodes**.

For each episode:

1. Start from the patient's current health state.
2. Select an action using ϵ -greedy selection.
3. Observe the reward.
4. Move to the next health state.
5. Update the Q-table.
6. Continue until the episode terminates.

Convergence Criterion

The algorithm can be considered converged when:

Maximum change in Q-values < 0.001 for several consecutive episodes.

This indicates that the Q-table is no longer changing significantly.

(c) Final Learned Policy

The final policy is obtained using:

$$\pi(s) = \operatorname{argmax}_a Q(s,a)$$

Example:

Patient State Best Action

Low Risk Monitor

Moderate Risk Diet

High Risk Exercise

Uncontrolled Medication/Clinical intervention

Improved Monitor

The exact final policy depends on the learned Q-table.

Ethical Considerations

A Reinforcement Learning system for medical treatment must be used very carefully.

1. Patient Safety

The system should not independently prescribe or change medication without qualified medical supervision.

2. Bias

If the training data under-represents certain patient groups, the model may produce biased recommendations.

3. Explainability

Doctors should be able to understand why an action was recommended.

4. Human Oversight

The AI should support doctors rather than replace clinical judgement.

5. Privacy

Patient information must be protected and handled according to applicable healthcare data-protection requirements.

6. Clinical Validation

The RL agent must be tested and validated extensively before deployment.

7. Safe Exploration

Unlike games, an AI agent cannot freely experiment with real patients. Exploration should therefore occur using simulations, historical data or carefully controlled clinical studies.

Overall Conclusion

The proposed system combines **Inductive Learning, Decision Trees, Statistical Learning and Reinforcement Learning** for diabetes management.

The Decision Tree provides interpretable diagnostic rules, while Logistic Regression provides a statistically interpretable risk score. Class imbalance should be handled carefully because detecting diabetic patients is more important than simply maximising accuracy. Finally, Q-Learning can theoretically learn personalised treatment policies, but a real healthcare deployment requires strong safety controls, clinical validation, explainability, privacy protection and human medical supervision.